Aula Completa sobre React

Da História aos Hooks e Ecossistema Moderno

O que é React?

- Uma biblioteca JavaScript para construir interfaces de usuário (UI).
- Criada e mantida pelo Facebook.
- Focada na construção de **componentes** de UI reutilizáveis.
- Não é um framework completo (como Angular), mas a base para muitos.

Princípios Chave:

- **Declarativo:** Descreva como sua UI deve parecer em qualquer estado, e o React cuidará de atualizá-la eficientemente quando os dados mudarem.
- **Baseado em Componentes:** Crie componentes encapsulados que gerenciam seu próprio estado e, em seguida, componha-os para criar UIs complexas.
- **Aprenda uma vez, escreva em qualquer lugar:** O React pode ser usado para construir aplicações web, mobile (com React Native) e desktop.

História e Evolução

- **2011:** Criado por Jordan Walke, um engenheiro de software do Facebook.
- **2012:** Implementado no feed de notícias do Facebook e, posteriormente, no Instagram.
- **Maio de 2013:** O React se torna open-source.
- **2015:** Lançamento do **React Native**, permitindo o desenvolvimento de aplicativos móveis com React.
- **2017 (React 16 - "Fiber"):** Uma reescrita completa do núcleo do React, permitindo renderização incremental e melhorando o desempenho.
- **2019 (React 16.8):** Introdução dos **Hooks**, revolucionando a forma como escrevemos componentes.
- **2022 (React 18):** Introduz recursos de concorrência (concurrency) para melhorar a experiência do usuário em aplicações complexas.

Conceitos Fundamentais

Virtual DOM

- O React cria uma cópia do DOM (Document Object Model) em memória.
- Quando o estado de um componente muda, o React atualiza o Virtual DOM.
- Em seguida, ele compara o Virtual DOM atualizado com a versão anterior e calcula a maneira mais eficiente de atualizar o DOM real.
- Isso evita manipulações diretas e lentas do DOM, resultando em melhor desempenho.

JSX (JavaScript XML)

- Uma extensão de sintaxe para JavaScript que se assemelha a HTML.
- Permite escrever a estrutura da UI dentro do código JavaScript.
- Não é obrigatório, mas é a forma mais comum e recomendada de escrever componentes React.

Exemplo de JSX:

```
const elemento = <h1>Olá, Mundo!</h1>;
```

Componentes: A Base de Tudo

Componentes Funcionais (Functional Components)

A maneira moderna e recomendada de escrever componentes React.

```
function BemVindo(props) {
  return <h1>Olá, {props.nome}</h1>;
}

// Uso do componente
<BemVindo nome="Maria" />;
```

Componentes de Classe (Class Components)

A forma mais antiga, ainda encontrada em bases de código legadas.

```
class BemVindo extends React.Component {
  render() {
    return <h1>Olá, {this.props.nome}</h1>;
  }
}
```

Props e State

Props (Propriedades)

- São como argumentos de uma função.
- Permitem passar dados de um componente pai para um componente filho.
- São **somente leitura** (imutáveis).

```
// Componente Pai
<PerfilUsuario nome="João" idade={30} />;

// Componente Filho (PerfilUsuario.js)
function PerfilUsuario(props) {
  return (
    <div>
      <p>Nome: {props.nome}</p>
      <p>Idade: {props.idade}</p>
    </div>
```

State (Estado)

- Dados que um componente gerencia internamente.
- Quando o estado de um componente muda, o React o renderiza novamente.
- É gerenciado usando o Hook `useState`.

React Hooks: Superpoderes para Componentes Funcionais

Hooks são funções que permitem que você "se conecte" aos recursos de estado e ciclo de vida do React a partir de componentes funcionais.

Regras dos Hooks

1. **Chame Hooks apenas no nível superior:** Não chame Hooks dentro de loops, condicionais ou funções aninhadas.
2. **Chame Hooks apenas em componentes de função React:** Não chame Hooks em funções JavaScript comuns.

Hooks Essenciais

`useState`

Permite adicionar estado a um componente funcional.

- **Declaração:** `const [estado, setEstado] = useState(valorInicial);`
- **Retorna:** Uma variável com o valor do estado e uma função para atualizá-lo.

```
import React, { useState } from 'react';

function Contador() {
  const [contagem, setContagem] = useState(0);

  return (
    <div>
      <p>Você clicou {contagem} vezes</p>
      <button onClick={() => setContagem(contagem + 1)}>Clique aqui</button>
    </div>
```

Hooks Essenciais (Continuação)

`useEffect`

Permite executar efeitos colaterais (side effects) em componentes funcionais. Exemplos: buscar dados de uma API, definir assinaturas (subscriptions) ou manipular o DOM manualmente.

- **Sintaxe:** `useEffect(() => { /* efeito */ }, [dependencias]);`
- O array de dependências controla quando o efeito é executado.
  - `[]`: Executa apenas uma vez, após a montagem do componente.
  - `[variavel]`: Executa sempre que `variavel` mudar.
  - Sem array: Executa a cada renderização (cuidado com loops infinitos!).

```jsx
import React, { useState, useEffect } from 'react';

function ExemploAPI() {
  const [dados, setDados] = useState(null);

  useEffect(() => {
    fetch('[https://api.exemplo.com/dados](https://api.exemplo.com/dados)')
      .then((res) => res.json())
      .then((data) => setDados(data));
  }, []); // O array vazio garante que o efeito só rode uma vez

  return <div>{dados ? dados.titulo : 'Carregando...'}</div>;
}
```

Hooks Essenciais (Continuação)

`useContext`

Permite acessar dados de um "Contexto" React sem precisar passar props por múltiplos níveis de componentes (prop drilling).

- **Uso:** Consumir um valor de um `React.createContext`.

```jsx
// 1. Criar o Contexto (em um arquivo separado)
const TemaContexto = React.createContext('claro');

// 2. Prover o Contexto (no componente pai)
function App() {
  return (
    <TemaContexto.Provider value="escuro">
      <BarraDeFerramentas />
    </TemaContexto.Provider>
  );
}

// 3. Consumir o Contexto (em qualquer componente filho)
function Botao() {
  const tema = useContext(TemaContexto);
  return <button className={tema}>Botão</button>;
}
```

Hooks Adicionais

- **`useReducer`**: Uma alternativa ao `useState` para gerenciar estados complexos, geralmente com um objeto de estado com múltiplas propriedades ou quando o próximo estado depende do anterior.
- **`useCallback`**: Retorna uma versão "memoizada" de uma função. A função só é recriada se uma de suas dependências mudar. Útil para otimização.
- **`useMemo`**: Retorna um valor "memoizado". Recalcula o valor apenas quando uma de suas dependências muda. Útil para otimizar cálculos caros.
- **`useRef`**: Fornece uma maneira de acessar nós do DOM diretamente ou de manter um valor mutável que não causa uma nova renderização quando atualizado.
- **`useLayoutEffect`**: Idêntico ao `useEffect` em sua assinatura, mas dispara de forma síncrona após todas as mutações do DOM.

O Ecossistema Moderno do React

Usar o React puro é possível, mas o desenvolvimento moderno geralmente envolve ferramentas e frameworks que fornecem uma experiência de desenvolvimento superior, otimizações e funcionalidades adicionais.

Vite

- Uma ferramenta de build extremamente rápida.
- Oferece um servidor de desenvolvimento quase instantâneo com Hot Module Replacement (HMR).
- Ótima escolha para iniciar projetos React de forma rápida e moderna.
- **Como usar:** `npm create vite@latest meu-app-react -- --template react`

Next.js (O Framework React para Produção)

- Um framework completo construído sobre o React.
- **Principais recursos:**
  - **Renderização Híbrida:** Server-Side Rendering (SSR) e Static Site Generation (SSG).
  - **Roteamento baseado em arquivos:** Crie páginas simplesmente adicionando arquivos na pasta `pages` ou `app`.
  - **Otimização de Imagens, Fontes e Scripts.**
  - **API Routes:** Crie endpoints de API como parte do seu projeto Next.js.
- Ideal para aplicações web completas, sites de e-commerce, blogs e qualquer projeto que precise de SEO e performance.
- **Como usar:** `npx create-next-app@latest`

Remix

- Outro framework full-stack focado nos fundamentos da web.
- **Principais recursos:**
  - **Carregamento de dados focado em rotas:** Cada rota pode definir suas próprias necessidades de dados.
  - **Mutações de dados simplificadas:** Formulários HTML funcionam como o esperado, com aprimoramentos de JavaScript.
  - **Nested Routing:** Rotas aninhadas que permitem layouts complexos.
- Excelente para aplicações web ricas em dados e focadas na experiência do usuário.
- **Como usar:** `npx create-remix@latest`

Outras Ferramentas

- **Create React App (CRA):** A forma "clássica" de iniciar projetos React. Embora ainda funcional, hoje a recomendação oficial do React é usar um framework como Next.js ou Remix para novos projetos.
- **Gatsby:** Um gerador de sites estáticos baseado em React, ótimo para blogs e sites de conteúdo.

Conclusão

- React revolucionou o desenvolvimento de interfaces de usuário com seu modelo de componentes e o Virtual DOM.
- Os **Hooks** modernizaram a forma de escrever componentes, tornando o código mais limpo e reutilizável.
- O ecossistema ao redor do React, com ferramentas como **Vite**, **Next.js** e **Remix**, oferece soluções poderosas para construir desde simples páginas até aplicações web complexas e de alto desempenho.